

Q1	Q2	Q3	Q4	Q5	Q6	Q7	Total (out of 25)
3.0	3.0	3.0	3.0	3.0	3.0	4.0	22.0

jh3397_ipynb

May 3, 2023

0.1 MA20278 Coursework - Jason Harley

```
[1]: # Packages, given functions and tests

import numpy as np
from matplotlib import pyplot as plt
import matplotlib.image as mpimg
import random
import pytest

def run_tests():
    pytest.main(args=['-v', '--tb=short', '--color=yes'])

def syntheticY(n):
    """
    Synthetic rank-2 n x n matrix
    """
    U = np.vstack((np.arange(n)/n, (1-np.arange(n)/n)**2)).T
    W = np.vstack((np.arange(n)/n, (1-np.arange(n)/n)**2)).T
    return U @ W.T

def bernoulli(n,p):
    """
    Bernoulli sampling.
    Produces a set of index pairs  $I = \{(i_1, i_2)\}$  where
     $i_1, i_2 = 0, \dots, n-1$ , and each pair is sampled with
    probability  $p$ 
    """
    I1, I2 = np.meshgrid(np.arange(n), np.arange(n))
    I1 = I1.flatten()
    I2 = I2.flatten()
    I = []
    for i1, i2 in zip(I1, I2):
        if (np.random.rand() < p):
            I.append([i1, i2])
    return np.array(I)

def Loss(U, W, Y, I):
    """
    Loss function (2), where
    U, W are n x r factor matrices,
    Y is a n x n data matrix,
```

```

and I is a m x 2 matrix of index pairs
"""

L = 0
for i1,i2 in I:
    L = L + (U[i1] @ W[i2] - Y[i1,i2])**2
return L / I.shape[0]

def initial(n,r):
    """
    Initial guess of the matrix factors
    """

    U0 = np.vstack((np.identity(r), np.zeros((n-r,r))))
    W0 = np.vstack((np.identity(r), np.zeros((n-r,r))))
    return U0, W0

def test_pointwise_gradient_1():
    """
    Unit tests for the pointwise gradient
    """

    Y = syntheticY(4)
    U,W = initial(4,2)
    vu = np.array([0,-1.125])
    vw = np.array([-1.125,0])
    assert np.array_equal(pointwise_gradient(U,W,Y,0,1), [vu, vw])

def test_pointwise_gradient_2():
    Y = syntheticY(4)
    U,W = initial(4,2)
    vu = np.array([0,0])
    vw = np.array([-0.5,0])
    assert np.array_equal(pointwise_gradient(U,W,Y,0,2), [vu, vw])

def test_pointwise_gradient_3():
    Y = syntheticY(4)
    U,W = initial(4,2)
    vu = np.array([-0.5,0])
    vw = np.array([0,0])
    assert np.array_equal(pointwise_gradient(U,W,Y,2,0), [vu, vw])

def test_full_gradient_1():
    """
    Unit tests for the full gradient
    """

    Y = syntheticY(4)
    U,W = initial(4,2)
    Gu = np.array([[0,-1.125], [0, 0], [0,0], [0,0]])
    Gw = np.array([[0, 0], [-1.125,0], [0,0], [0,0]])
    assert np.array_equal(full_gradient(U,W,Y, np.array([[0,1]])), [Gu, Gw])

def test_full_gradient_2():
    Y = syntheticY(4)

```

```

U,W = initial(4,2)
Gu = np.array([[0, 0], [-9/16,0], [0, 0], [0,0]])
Gw = np.array([[0,-9/16], [0, 0], [-1/4,0], [0,0]])
assert np.array_equal(full_gradient(U,W,Y, np.array([[1,0], [0,2]])),
↪ [Gu, Gw])

def test_full_gradient_3():
    Y = syntheticY(4)
    U,W = initial(4,2)
    Gu = np.array([[0,-9/32], [-9/32,0], [-1/8,0], [0,0]])
    Gw = np.array([[0,-9/32], [-9/32,0], [0, 0], [0,0]])
    assert np.array_equal(full_gradient(U,W,Y, np.array([[1,0], [0,1],
↪ [2,0], [0,0]])), [Gu, Gw])

```

Q1: pointwise loss gradient. See below my code for the pointwise gradient function

✓ [2]:

```

def pointwise_gradient(U,W,Y,i1,i2):
    vu = 2*(U[i1]@W.T[:,i2] - Y[i1,i2])*W[i2]
    vw = 2*(U[i1]@W.T[:,i2] - Y[i1,i2])*U[i1]
    return vu, vw

```

Q2: full gradient. See below my code for the full gradient function

✓ [3]:

```

def full_gradient(U,W,Y,I):
    m = len(I)
    n,r = U.shape
    Gu = np.zeros(n*r).reshape(n,r) # Initialise Gu and Gw as nxr matrices
    ↪ filled with zeros
    Gw = np.zeros(n*r).reshape(n,r)
    for [i1,i2] in I:
        vu, vw = pointwise_gradient(U,W,Y,i1,i2) # Computing the pointwise
    ↪ loss (using the function in Q1) for all index pairs in 'I'
        Gu[i1] += vu # Incrementing the rows
        Gw[i2] += vw
    return Gu/m , Gw/m

```

[4]: run_tests()

```

===== test session starts

=====
platform win32 -- Python 3.9.13, pytest-7.1.2, pluggy-1.0.0 --
C:\Users\jason\anaconda3\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\jason\ML Python\CW
plugins: anyio-3.5.0
collecting ... collected 0 items

===== no tests ran in 0.01s

=====

```

All tests are passed - indicating the functions are working properly

Q3: gradient descent. See below my code for the gradient descent function

```
[4]: def gd(U0, W0, Y, I, Itest, t=1, eps=1e-6, K=100):
    U = U0
    W = W0
    Losses = [] # Initialising the list of losses as an empty matrix - of
    ↪which to later append to
    for k in range(K):
        if Loss(U,W,Y,Itest) < eps: # If the algorithm converges the
        ↪for-loop will be interrupted
            break
        Gu, Gw = full_gradient(U,W,Y,I) # Calculating the full gradient
        ↪(using the function in Q2)
        U -= t*Gu # Updating the iterates
        W -= t*Gw
        Losses = np.append(Losses, Loss(U,W,Y,Itest)) # Appending to the
        ↪loss matrix
        print('k = ' + str(k)) # Observing what iteration the algorithm
        ↪converges at
        plt.plot(range(len(Losses)), Losses, '-') # Using pyplot to plot the
        ↪loss as a function of k
        plt.yscale('log')
        plt.xlabel('Iteration number (k)')
        plt.ylabel('Log ( Test loss values )')
        plt.title('Logarithmic plot of Loss against k')
        plt.show()
    return U, W, Losses # The loss array is also returned for later use
```

Q4: testing GD on full data. See below my code for testing the gradient descent function

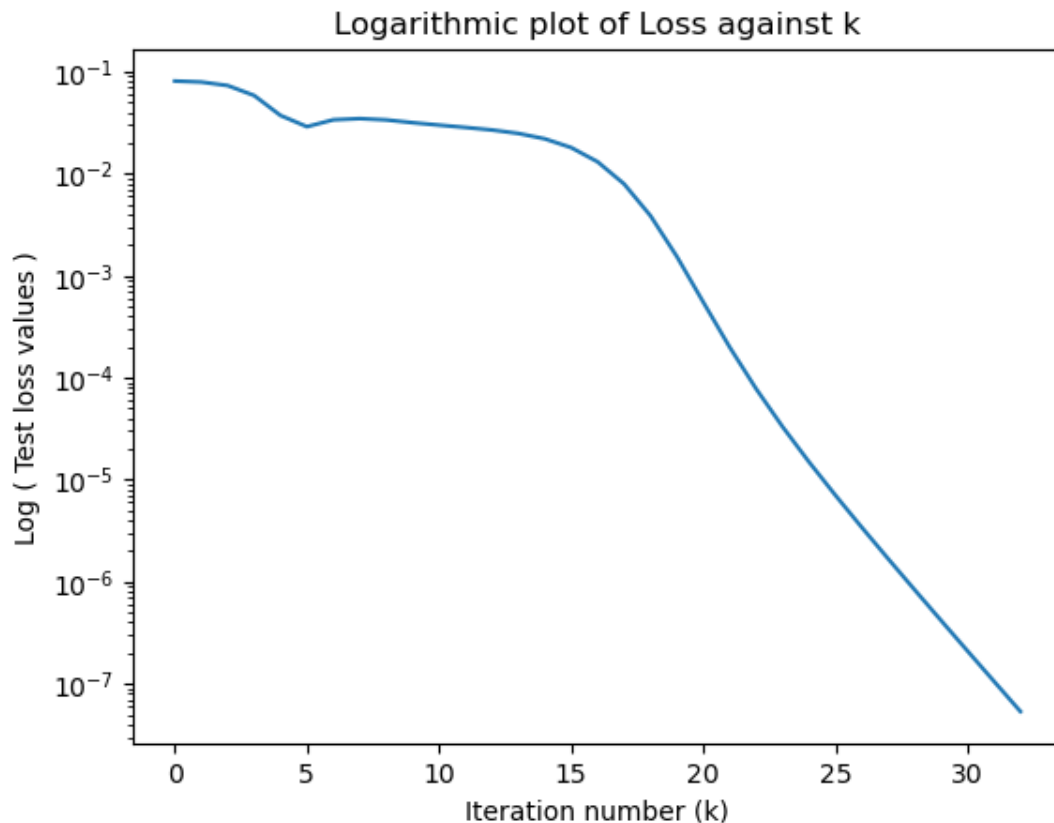
```
[6]: Y = syntheticY(32) # Using the given functions to generate the required
    ↪matrices
    Iext = bernoulli(32,1)
    U0, W0 = initial(32,2)

    pairs = random.sample(range(len(Iext)), 20) # Using the random sample
    ↪function to determine which random pairs to use (from Iext) in Itest
    Itest = Iext[pairs] # Selecting the randomly chosen pairs from Iext
    I = np.delete(Iext, pairs, 0) # Computing 'I' as the set complement of Iext
    ↪and Itest
```

1e-8

```
[7]: U, W, Losses = gd(U0, W0, Y, I, Itest, t=32, eps=10e-8, K=5000) # Code to
    ↪run algorithm, collect losses and plot graph of convergence
```

k = 33



```
[8]: Relative_Error = np.linalg.norm(U@W.T - Y) / np.linalg.norm(Y) # Code to
      ↪ compute relative error and final test loss
      Final_Test_Loss = Losses[-1]
      print('Relative error = ' + str(Relative_Error))
      print('Final test loss = ' + str(Final_Test_Loss))
```

Relative error = 0.0003348388719024694
 Final test loss = 5.3496817571199444e-08

Q5: stochastic average gradient. See below my code for the stochastic average gradient (SAG) function

```
[5]: def sag(U0, W0, Y, I, Itest, t=1, eps=1e-6, K=100):
      U = U0 # Initialising the matrices/arrays (similarly to as in Q2/3)
      W = W0
      m = len(I) # It is important that m is not fixed as it will change based
      ↪ on the bernoulli undersampling factor (p)
      Vu_ = np.zeros((m, len(U.T))) # Underscore in names to represent tilde
      Vw_ = np.zeros((m, len(U.T)))
      Gu_ = np.zeros((U.shape))
      Gw_ = np.zeros((U.shape))
      Losses = []
      for k in range(K):
          if Loss(U,W,Y,Itest) < eps:
```

✓

```

        break
    i = random.randint(0,len(I)-1) # Choosing a random index pair
    i1,i2 = I[i]
    Gu_[i1] -= Vu_[i]/m # Subtracting the old gradients
    Gw_[i2] -= Vw_[i]/m
    Vu_[i], Vw_[i] = pointwise_gradient(U,W,Y,i1,i2) # Calculating the
    ↪new gradients
    Gu_[i1] += Vu_[i]/m # Adding the new gradients
    Gw_[i2] += Vw_[i]/m
    U -= t*Gu_ # Updating the iterates
    W -= t*Gw_
    Losses = np.append(Losses, Loss(U,W,Y,Itest))
    print('k = ' + str(k))
    plt.plot(range(len(Losses)), Losses, '-')
    plt.yscale('log')
    plt.xlabel('Iteration number (k)')
    plt.ylabel('Log ( Test loss values )')
    plt.title('Logarithmic plot of Loss against k')
    plt.show()
    return U, W, Losses

```

Q6: testing SAG on full data. Using the same data-generating method as in Q4 we test the stochastic gradient descent function (see below):

```

[10]: Y = syntheticY(32) # Using the same generating method as in Q4
      Itest = bernoulli(32,1)
      U0, W0 = initial(32,2)

      pairs = random.sample(range(len(Itest)), 20)
      ↪Itest = Itest[pairs]
      I = np.delete(Itest, pairs, 0)

```

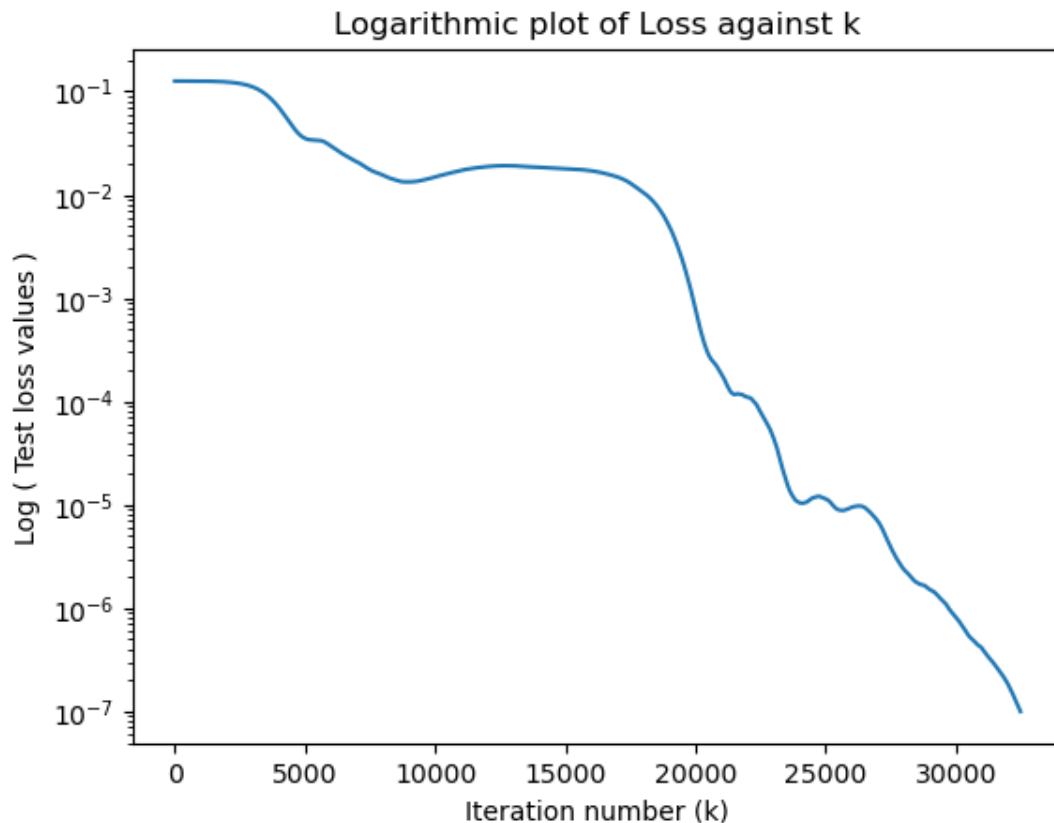
1e-8

```

[11]: U, W, Losses = sag(U0, W0, Y, I, Itest, t=1/16, eps=10e-8, K=50000) # Code
      ↪to run algorithm, collect losses and plot graph of convergence

```

k = 32486



```
[12]: Relative_Error = np.linalg.norm(U@W.T - Y) / np.linalg.norm(Y) # Code to
      ↪ compute relative error and final test loss
      Final_Test_Loss = Losses[-1]
      print('Relative error = ' + str(Relative_Error))
      print('Final test loss = ' + str(Final_Test_Loss))
```

Relative error = 0.0009858773844871043
 Final test loss = 9.999199432499666e-08

Q7: tests with real data. In order to find L_{low} I first computed the data (as required) and then used it alongside the matrix Y from the data file in both algorithms, collecting the minimum losses (minimum value in losses array) across multiple runs (at all learning rates) and then manually computing the averages (see code below):

```
[13]: data = np.load('CW.npz') # First the data must be loaded from the given file
      Y = data['Y'] # Defining a python-usable matrix from the data
      n = len(Y)

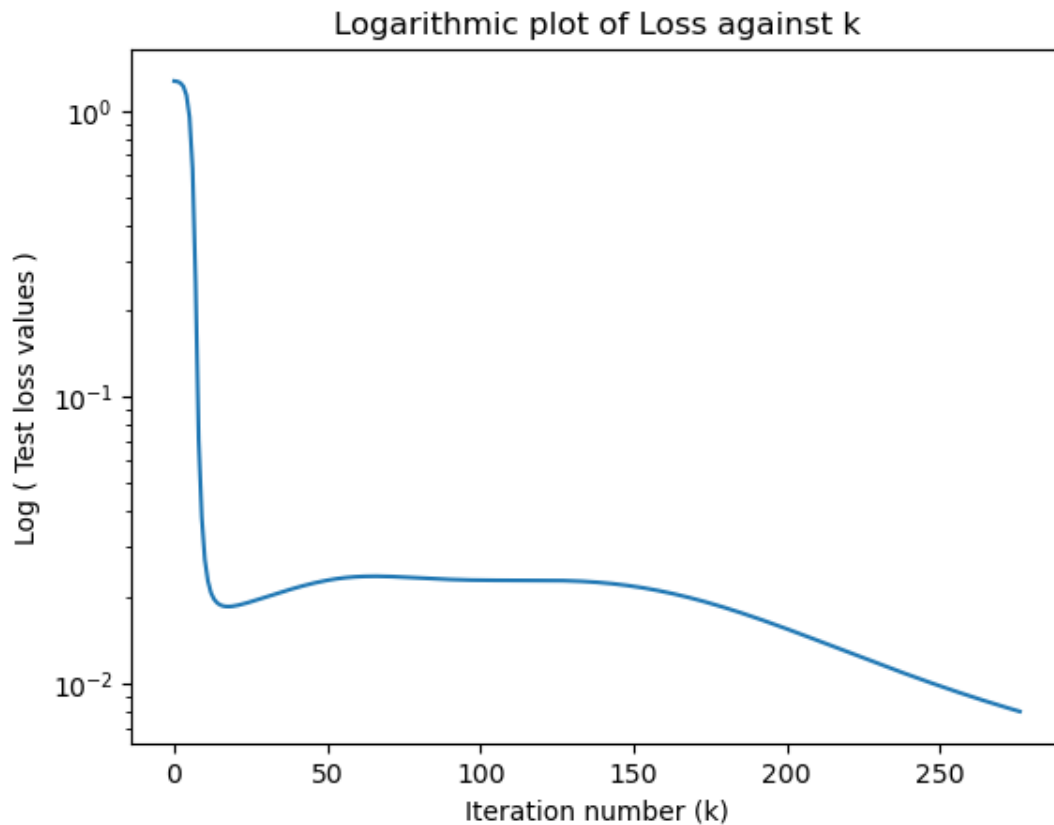
      Iext = bernoulli(n,1/3) # Generating the index pairs and initial guesses
      U0, W0 = initial(n,r=4)

      pairs = random.sample(range(len(Iext)), 20)
      Itest = Iext[pairs]
      I = np.delete(Iext, pairs, 0)
```

Using this method I have determined that $\min_{U,W} L_{I_{test},Y}(U,W) \approx L_{low} = 3.72e-3 \approx 4e-3$ (see full table in report PDF) thus we set $\epsilon = 8e-3$, , ϵ and find the learning rate that gives fastest convergence for each algorithm (after re-initialising the data inbetween), just like in Q4 and Q6 (except with increased K in order to achieve convergence):

[14]: `U_gd, W_gd, Losses = gd(U0, W0, Y, I, Itest, t=16, eps=8e-3, K=5000) # Code`
↪to run algorithm, collect losses and plot graph of convergence

k = 277



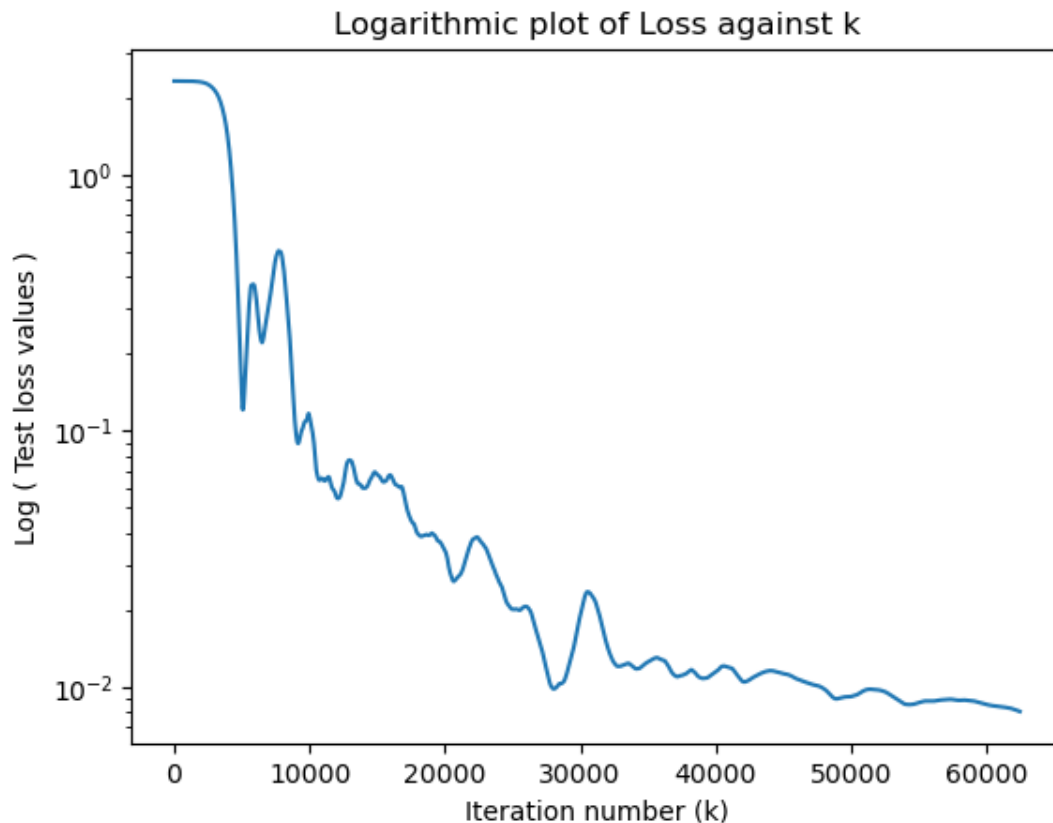
[15]: `data = np.load('CW.npz') # Re-initialising the data (nothing new here)`
`Y = data['Y']`
`n = len(Y)`

`Itest = bernoulli(n,1/3)`
`U0, W0 = initial(n,r=4)`

`pairs = random.sample(range(len(Itest)), 20)`
`Itest = Itest[pairs]`
`I = np.delete(Itest, pairs, 0)`

`U_sag, W_sag, Losses = sag(U0, W0, Y, I, Itest, t=1/16, eps=8e-3, K=500000)`
↪# Code to run algorithm, collect losses and plot graph of convergence

k = 62469



```
[16]: Relative_Error = np.linalg.norm(U_sag@W_sag.T - Y) / np.linalg.norm(Y) #_
      ↪Code to compute relative error and final test loss
Final_Test_Loss = Losses[-1]
print('Relative error = ' + str(Relative_Error))
print('Final test loss = ' + str(Final_Test_Loss))
```

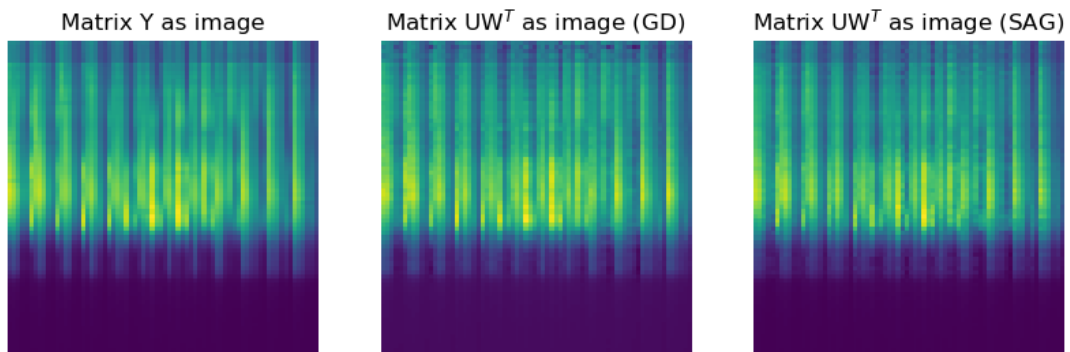
Relative error = 0.062080256313707075
Final test loss = 0.007999701456472143

After carrying out this method I have determined that for $\epsilon = 2 \times L_{low}$ the GD algorithm converges fastest for $t = 16$ and the SAG algorithm converges fastest for $t = 1/16$ (full table in PDF report)

See below code for plot Y and UW.T as images: (for convenience the images are plotted for the fastest converging t in each algorithm as matrix accuracy is very similar across all converging learning rates; the code to produce the matrices is above).

```
[17]: fig = plt.figure(figsize=(10,6))
      fig.add_subplot(1,3,1)
      plt.imshow(Y)
      plt.axis('off')
      plt.title('Matrix Y as image')
      fig.add_subplot(1,3,2)
      plt.imshow(U_gd@W_gd.T) #U@W.T for GD algorithm (computed using t=16 above)
      plt.axis('off')
```

```
plt.title('Matrix  $UW^T$  as image (GD)')
fig.add_subplot(1,3,3)
plt.imshow(U_sag@W_sag.T) # $U@W.T$  for SAG algorithm (computed using  $t=1/16$ 
↪above)
plt.axis('off')
plt.title('Matrix  $UW^T$  as image (SAG)')
plt.show()
```



Visually we do not see much difference between the images, indicating a good approximation, and a similar performance from GD and SAG.